📕 คู่มือสถาปัตยกรรมข้อมูลฉบับสมบูรณ์: Designing Data-Intensive Applications (DDIA) x Omni-Flow Labs

เอกสารสรุปทฤษฎี กฎเหล็ก (Golden Rules) และการจับคู่การทดลองปฏิบัติจริง (Hands-on Labs) ในโปรเจกต์ **Omni-Flow** เข้ากับเนื้อหาทั้ง 12 บทของหนังสือ *Designing Data-Intensive Applications (DDIA)* โดย Martin Kleppmann สำหรับเตรียมสอบสัมภาษณ์ระดับ Senior Data Architect / Toptal

---

🏛️ ภาพรวมการจับคู่: บทเรียนใน DDIA ⟶ ชุดการทดลองใน Omni-Flow

บทในหนังสือ DDIA	หัวใจสำคัญของบท	กฎเหล็ก (DDIA Golden Rules)	ตรงกับ Lab / การทดสอบใน Omni-Flow
Ch. 1: Reliable, Scalable, Maintainable	ความทนทานต่อความผิดพลาด (Fault Tolerance) และการวัด Load ด้วย Percentile	ห้ามให้ **Fault (จุดเสียย่อย)** กลายเป็น **Failure (ระบบล่มทั้งก้อน)**	**Command Center (Scenario 2 & 4):** ทดสอบระบบไม่ล่มเมื่อ Consumer ขาดการเชื่อมต่อ 3 วินาที และทดสอบ Stress Test 1,500+ TPS
Ch. 3: Storage & Retrieval (OLTP vs OLAP)	โครงสร้าง B-Trees (Row-oriented OLTP) vs LSM-Trees & Columnar Storage (OLAP)	ห้ามรัน Analytic Query หนักๆ บนตาราง OLTP เด็ดขาด ให้แยก Columnar OLAP เสมอ	**Lab 2 & Lab 3:** เปรียบเทียบ **PostgreSQL** (Row B-Tree) vs **ClickHouse** (`MergeTree` Columnar) vs **Parquet** บน **MinIO S3**
Ch. 4: Encoding & Evolution	รูปแบบการเก็บข้อมูล (JSON vs Binary Columnar Parquet) และ Schema Evolution	ข้อมูลที่เขียนลงดิสก์/สตรีมต้องรองรับทั้ง **Forward & Backward Compatibility**	**Lab 2 (MinIO Lakehouse):** เปรียบเทียบ JSON ใน Kafka กับไฟล์ `.parquet` ที่มี Schema ฝังใน Footer บน S3
Ch. 5: Replication	Leader-based Replication, Synchronous vs Asynchronous, และ Replication Lag	**Read-after-write / Monotonic Reads:** ผู้อ่านต้องเห็นข้อมูลที่ Commit ครบตาม Quorum แล้วเท่านั้น	**Command Center:** การตั้งค่า `acks=all`, การตรวจสอบ **In-Sync Replicas (ISR)** และ **High Watermark Offset**
Ch. 6: Partitioning (Sharding)	การแบ่งข้อมูลเพื่อขยายสเกล, การเลือก Partition Key และการแก้ปัญหา **Hot Spot / Data Skew**	Ordering รับประกันเฉพาะใน Partition เดียวกัน; ถ้าเกิด Hot Key ต้องแก้ด้วย **Key Salting**	**Command Center (Scenario 1, 5A, 5B):** เทียบ Entity Key (`device_id`) vs Hot-Spot Skew (`P0` 90%) vs Key Salting (`#salt-0..2`)
Ch. 7 & 8: Transactions & Distributed Trouble	ปัญหาเครือข่ายกระตุก (Network Partition), GC Pause, และการส่งซ้ำ (At-Least-Once)	เครือข่ายไม่มีวันเสถียร 100% ระบบปลายทางต้องเป็น **Idempotent (รับซ้ำได้แต่ผลลัพธ์ไม่เบิ้ล)**	**Command Center (Scenario 3):** เปิด `enable.idempotence=True` + ทดสอบยิงซ้ำเข้า ClickHouse `ReplacingMergeTree(FINAL)`

Ch. 9: Consistency & Consensus	Linearizability, Total Order Broadcast, และอัลกอริทึมฉันทามติ (**Raft / Paxos**)	ทุกการเลือกตั้ง Leader ใหม่ต้องมีเลขยุค (**Epoch / Fencing Token**) เพิ่มขึ้นเสมอเพื่อกัน Split-Brain	**ทุก Lab (Kafka KRaft Mode):** ใช้ Raft Quorum (`CONTROLLER` role) แทน ZooKeeper
Ch. 10: Batch Processing	การประมวลผลข้อมูลก้อนใหญ่แบบ Immutable และ **Batch Reconciliation / Audit**	**"If the output is deterministic and inputs are immutable, any bug can be fixed by re-running"**	**Command Center (Bottom Row) & Lab 2:** การกระทบยอด (`Source Sent == ClickHouse FINAL`) และการทำ Time-Travel บนไฟล์ Parquet
Ch. 11: Stream Processing	Event Streams, **Change Data Capture (CDC)**, Windowing, และ Stream-Table Joins	Database คือผลลัพธ์ของการสะสม Event Stream (**Log-centric / Kappa Architecture**)	**Lab 1 (Apache Flink)**, **Lab 3 (Debezium CDC)**, และ **Command Center (Stateful Worker)**
Ch. 12: The Future of Data Systems	การผสานระบบหลายชิ้นเข้าด้วยกัน (Derived Data) และการทำ **End-to-End Auditability**	อย่าเชื่อใจว่าท่อส่งข้อมูลจะไม่พัง ให้สร้างระบบ **Continual Auditing (ตรวจสอบยอดอัตโนมัติ)** เสมอ	**Command Center (Batch Audit Check):** ตรวจเช็ก `Diff = 0` แบบเรียลไทม์ตลอดเวลา

🔍 เจาะลึกรายบท: ทฤษฎี + กฎในหนังสือ + วิธีทดสอบในโปรเจกต์

บทที่ 1: Reliable, Scalable, and Maintainable Applications
📖 เนื้อหาที่หนังสือพูดถึง
1. **Reliability (ความเชื่อถือได้):** ระบบต้องทำงานถูกต้องแม้เกิดเหตุขัดข้อง (Fault) โดยหนังสือแยกคำว่า **Fault** (ชิ้นส่วนหนึ่งมีปัญหา เช่น เน็ตหลุด 3 วินาที หรือ Consumer ตาย) ออกจากคำว่า **Failure** (ระบบพังจนให้บริการผู้ใช้ไม่ได้)
2. **Chaos Engineering:** การจงใจฉีดความผิดพลาดเข้าระบบ (เหมือน Netflix Chaos Monkey) เพื่อพิสูจน์ว่าระบบฟื้นตัวเองได้จริง
3. **Scalability (การรองรับการขยายตัว):** การวัดโหลดด้วย **Throughput (TPS / Records per sec)** และความหน่วง (**Latency / Lag**)

⚖️ กฎเหล็กประจำบท (DDIA Law #1)
> *"เราไม่สามารถทำให้โอกาสเกิด Fault เป็นศูนย์ได้ แต่เราต้องออกแบบสถาปัตยกรรมให้ทนต่อ Fault (Fault-Tolerant) เพื่อป้องกันไม่ให้ Fault ลุกลามกลายเป็น System Failure"*

🧪 การทดสอบในโปรเจกต์ของเรา (`omni-command-center`)
* **กดปุ่ม `⚡ 2. Network Gap (3s Drop)`:** เป็นการทำ Chaos Injection ตัดท่อฝั่งรับ 3 วินาที จะเห็นว่า Producer ไม่ล่ม (เพราะ Kafka รับหน้าเป็น Buffer แทน) และเมื่อสัญญาณกลับมา ระบบทำ **Offset Recovery** ดึงข้อมูลย้อนหลังกลับมาครบ 100% (`0 Loss`)

* **กดปุ่ม `🔥 4. Stress Test (1,500+ TPS)`:** ทดสอบขยายโหลดขึ้น 40 เท่า เพื่อดูพฤติกรรมของ `Consumer Lag` และการใช้ทรัพยากร `CPU / RAM`

บทที่ 3 & 4: Storage, Retrieval, and Encoding (OLTP vs OLAP & Columnar Parquet)
📖 เนื้อหาที่หนังสือพูดถึง
1. **OLTP (Online Transaction Processing):** เช่น **PostgreSQL** ใช้โครงสร้าง **B-Tree (Row-oriented)** เก็บข้อมูลทั้งแถวติดกัน เหมาะกับการค้นหา/แก้ไขทีละแถวตาม `order_id` (`INSERT / UPDATE / DELETE`)
2. **OLAP (Online Analytical Processing):** เช่น **ClickHouse (`MergeTree`)** และไฟล์ **Apache Parquet** ใช้โครงสร้าง **Column-oriented Storage** เก็บข้อมูลแยกตามคอลัมน์ ทำให้เวลาคำนวณ `SUM(amount)` หรือ `AVG(temperature)` จากข้อมูล 100 ล้านแถว ระบบจะอ่านเฉพาะไฟล์ของคอลัมน์นั้นขึ้นมาบีบอัดคำนวณใน CPU Cache (SIMD Vectorized Execution) เร็วกว่า OLTP เป็นพันเท่า!
3. **Self-Describing Columnar Encoding (`Parquet`):** ไฟล์ `.parquet` ฝังโครงสร้างตาราง (Schema) และสถิติ `Min/Max` ของแต่ละก้อนข้อมูล (Row Group) ไว้ที่ท้ายไฟล์ (Footer) ทำให้ทำ **Predicate Pushdown** (ข้ามไฟล์ที่ไม่มีข้อมูลที่ค้นหา) ได้ทันที

⚖️ กฎเหล็กประจำบท (DDIA Law #2)
> *"แยกภาระงาน OLTP (รับธุรกรรมลูกค้า) ออกจาก OLAP (วิเคราะห์ข้อมูล) เสมอ และเก็บข้อมูลระยะยาวในรูปแบบ Columnar (Parquet) บน Object Storage เพื่อความคุ้มค่าสูงสุด"*

🧪 การทดสอบในโปรเจกต์ของเรา (`lab2-minio-lakehouse` & `lab3-debezium-cdc`)
* **ใน `lab3-debezium-cdc`:** ใช้ **PostgreSQL** รับคำสั่ง `INSERT / UPDATE / DELETE` แล้วซิงก์ผ่าน Debezium ไปเข้า **ClickHouse** เพื่อทำ OLAP Analytics โดยไม่กวนเครื่อง PostgreSQL
* **ใน `lab2-minio-lakehouse`:** แพ็กข้อมูลทุกๆ 20 รายการเป็นไฟล์ `.parquet` เก็บลง **MinIO S3 (`http://localhost:9001`)** และใช้คำสั่ง `DESCRIBE TABLE s3(...)` ใน DBeaver เพื่อพิสูจน์ว่าไฟล์ `.parquet` มี Schema ฝังอยู่ในตัวไฟล์ พร้อมทำ **Time-Travel Query** ย้อนดูข้อมูลราย Snapshot ได้

บทที่ 5 & 9: Replication, Consistency, and Consensus (KRaft, ISR & Watermarks)
📖 เนื้อหาที่หนังสือพูดถึง
1. **Leader-Follower Replication:** ทุก Partition ใน Kafka จะมี **Leader 1 ตัว** คอยรับงานเขียน/อ่าน และมี **Followers** คอยดึง Log ไปสำเนาตามลำดับ
2. **In-Sync Replicas (ISR) & High Watermark:**
    * Follower ที่ตามทัน Leader จะอยู่ในกลุ่ม **ISR**
    * ตำแหน่งข้อมูลล่าสุดที่ทุกตัวในกลุ่ม ISR บันทึกลงดิสก์ครบแล้ว เรียกว่า **High Watermark Offset**

    * Consumer จะมองเห็นข้อมูลได้ไม่เกินจุด **High Watermark** เท่านั้น (ป้องกันการอ่านข้อมูลที่ยังไม่ถูก Commit หรือ Dirty Read หาก Leader ตายกะทันหัน)
3. **Consensus & KRaft (Raft Algorithm):**
    * Kafka ยุคใหม่ใช้ **KRaft (Kafka Raft)** ในการเลือกตั้ง Controller Leader โดยใช้เลขยุค (**Epoch / Term**) กำกับทุกคำสั่ง ทำหน้าที่เป็น **Fencing Token** ป้องกันปัญหา **Split-Brain** (Leader ตัวเก่าที่เน็ตค้างกลับมาสั่งงานซ้อนทับ Leader ตัวใหม่)

⚖️ กฎเหล็กประจำบท (DDIA Law #3)
> *"ความทนทานสูงสุด (Zero Data Loss on Leader Failover) เกิดจากการผสาน `acks = all` + การตรวจสอบกลุ่ม `ISR` + การใช้ `Epoch Fencing Token` ในอัลกอริทึม Raft"*

🧪 การทดสอบในโปรเจกต์ของเรา (`omni-command-center`)
* ดูที่กล่อง **`DDIA Partition Skew & ISR Status`** บนหน้า Dashboard (`http://localhost:8000`) ซึ่งดึงค่า Metadata จริงจาก Kafka Broker มาแสดงทั้ง **`Leader Broker ID`**, **`ISR Count`**, **`Committed Offset`** และ **`High Watermark Offset`** ของทั้ง 3 พาร์ติชัน (`P0, P1, P2`)

บทที่ 6: Partitioning (Sharding, Data Skew & Key Salting)
📖 เนื้อหาที่หนังสือพูดถึง
1. **Key-Hash Partitioning:** การนำค่า Key มาเข้าฟังก์ชัน Hash (`murmur2(key) % num_partitions`) เพื่อกำหนดว่าจะส่งข้อความไปลง Partition ไหน
2. **Causal Ordering Guarantee:** Kafka รับประกันลำดับก่อน-หลังของเหตุการณ์ **เฉพาะภายใน Partition เดียวกันเท่านั้น** ดังนั้นถ้าต้องการให้เหตุการณ์ของ `SENSOR-001` เรียงตามเวลาเสมอ ต้องใช้ `key = device_id`
3. **Data Skew / Hot Spot (Celebrity Problem):** หาก Key บางตัวมีข้อมูลมากกว่าตัวอื่นมหาศาล (เช่น เซ็นเซอร์หลักตัวเดียวส่งข้อมูล 90% ของทั้งโรงงาน) ข้อมูลจะไปกระจุกอยู่ที่ Partition เดียว (`P0`) ทำให้เครื่องนั้นคอขวด
4. **การแก้ปัญหาด้วย Key Salting:** การเติมตัวเลขสุ่มหรือลำดับ (`#salt-0..2`) ต่อท้าย Hot Key เพื่อกระจายข้อมูลลงหลาย Partition ให้เท่าๆ กัน (แลกกับการต้องมารวมยอดตอนคิวรีใน ClickHouse)

⚖️ กฎเหล็กประจำบท (DDIA Law #4)
> *"เลือก Partition Key ตามขอบเขตที่ต้องการรักษาลำดับเวลา (Entity ID) เสมอ แต่หากเกิด Hot Key จนพาร์ติชันเบ้ (Data Skew) ต้องทำ Key Salting เพื่อกระจายโหลด"*

🧪 การทดสอบในโปรเจกต์ของเรา (`omni-command-center`)
* **กดปุ่ม `🟢 1. Happy Path (Entity Key)`:** ใช้ `key = device_id` รักษาลำดับรายเครื่อง
* **กดปุ่ม `⚖️ 5A. Data Skew (Hot-Spot P0)`:** จำลองปัญหา Hot Key บังคับให้ข้อมูล 90% ไหลเข้า `Partition 0` จะเห็นหลอด `P0` บนหน้าเว็บเปลี่ยนเป็นสีส้มและพุ่งโด่งทันที

* **กดปุ่ม `🧂 5B. Key Salting (Balanced)`:** เติม `#salt-0..2` ต่อท้าย Key เกลี่ยโหลดให้ `P0, P1, P2` กลับมาสมดุลกันที่ ~33.3%

บทที่ 7, 8 & 11: Stream Processing, Idempotency, CDC & Kappa Architecture
📖 เนื้อหาที่หนังสือพูดถึง
1. **Delivery Semantics (At-Most-Once / At-Least-Once / Exactly-Once):**
   * ในระบบเครือข่ายจริง เมื่อเกิด Timeout ผู้ส่ง (`Producer`) ไม่รู้ว่าข้อความไปถึงหรือยัง จึงต้อง **Retry (ส่งซ้ำ)** ทำให้เกิดข้อมูลซ้ำในท่อ (**At-Least-Once Delivery**)
   * เพื่อให้ผลลัพธ์สุดท้ายถูกต้องเหมือนประมวลผลครั้งเดียว (**Effectively-Once / Idempotency**) ปลายทางต้องมีกลไก **Deduplication ตาม Unique Event ID**
2. **Change Data Capture (CDC):** การเปลี่ยนฐานข้อมูล OLTP ให้กลายเป็น Event Stream โดยอ่านจากไฟล์ **Write-Ahead Log (WAL)** ด้วยเครื่องมืออย่าง **Debezium**
3. **Stateful Stream Processing (Kappa Architecture):**
   * **Stream-Table Join:** การนำข้อมูลดิบใน Stream มาประกบกับตารางข้อมูลหลัก (Master Lookup Table) กลางอากาศ
   * **Windowing (Tumbling Window):** การตัดแบ่ง Stream ออกเป็นหน้าต่างเวลาคงที่ (เช่น ทุกๆ 5 วินาที หรือ 10 วินาที) เพื่อคำนวณค่าสถิติ
   * **Complex Event Processing (CEP):** การจำสถานะ (State) เพื่อตรวจจับรูปแบบเหตุการณ์ที่เกิดต่อเนื่องกัน

⚖️ กฎเหล็กประจำบท (DDIA Law #5)
> *"At-Least-Once Retry + Idempotent Sink (Deduplication) = Effectively-Once Processing (ข้อมูลไม่หาย และยอดไม่เบิ้ล)"*

🧪 การทดสอบในโปรเจกต์ของเรา
1. **การทดสอบ Idempotency (`omni-command-center` — Scenario 3):**
   * กดปุ่ม `🔁 3. Idempotency (Dup Retry)` ตัว `producer.py` จะจงใจยิง `event_id` เดิมซ้ำเข้า Kafka อีก 2 รอบทุกๆ 5 รายการ
   * ตาราง ClickHouse ใช้ `ReplacingMergeTree(ingested_at) ORDER BY (event_id)` เวลาคิวรีด้วย `FINAL` จะตัดแถวซ้ำออกทั้งหมด ทำให้ช่อง `FINAL` และผลรวมค่าไฟตรงกับต้นทาง 100% (`Diff = 0`)
2. **การทดสอบ Apache Flink (`lab1-flink-stream`):**
   * ใช้ **Apache Flink Cluster (`http://localhost:8081`)** รันคำสั่ง Flink SQL ทำ `TUMBLE(proc_time, INTERVAL '10' SECOND)` และกรองธุรกรรมเสี่ยงสูง (`Fraud >= 25,000 THB`) ก่อนส่งเข้า ClickHouse
3. **การทดสอบ Debezium CDC (`lab3-debezium-cdc`):**
   * ดักจับ `INSERT ('c')`, `UPDATE ('u')`, และ `DELETE ('d')` จาก PostgreSQL WAL Log ส่งเข้า Kafka แล้วซิงก์ลงตาราง ClickHouse `ReplacingMergeTree(version_ts, is_deleted)`

บทที่ 10 & 12: Batch Reconciliation & End-to-End Auditing

📖 เนื้อหาที่หนังสือพูดถึง

ในบทสุดท้ายของหนังสือ (Chapter 12: *Aiming for Correctness*) Martin Kleppmann เน้นย้ำว่า แม้เราจะตั้งค่า Kafka, Flink และ Database ดีแค่ไหน แต่ในระบบกระจายตัวขนาดใหญ่ วันหนึ่งอาจเกิด Bug ในโค้ดหรือฮาร์ดแวร์ได้เสมอ ดังนั้นระบบระดับองค์กรต้องมี **End-to-End Auditing / Batch Reconciliation** คอยตรวจทานยอดระหว่าง **"ต้นทางที่ส่ง"** กับ **"ปลายทางที่เก็บจริง"** ตลอดเวลา

⚖️ กฎเหล็กประจำบท (DDIA Law #6)

> *"Trust, but Verify — อย่าเชื่อแค่การตั้งค่าของท่อส่งข้อมูล แต่จงสร้างระบบกระทบยอด (End-to-End Reconciliation Check) เพื่อพิสูจน์ด้วยตัวเลขเสมอว่า `Source Total == Sink Total`"*

🧪 การทดสอบในโปรเจกต์ของเรา (`omni-command-center`)

* ในส่วน **Bottom Row (Batch Audit / Reconciliation Check)** ของหน้า Dashboard:
  * เทียบยอด `Source Sent` (จากตาราง `source_audit_kappa`) กับ `ClickHouse (FINAL)` (จากตาราง `sensor_events_kappa FINAL`) ทั้ง **จำนวนรายการ (Unique Event Count)** และ **ผลรวมพลังงาน (Total Energy kWh)**
  * เมื่อกดปุ่ม `⏸️ Pause (Audit)` เพื่อรอให้ข้อความก้อนสุดท้ายในท่อไหลเข้า ClickHouse (ใช้เวลา < 0.2 วินาที) จะเห็นว่า **`Diff (Gap) = 0`** และขึ้นป้ายรับรอง **`✅ RECONCILED (100% MATCH - 0 LOSS / 0 DUPLICATES)`**!